Submission Worksheet

Submission Data

Course: IT202-008-S2026
Assignment: IT202 - Milestone 3
Student: George C. (gec23)
Status: Submitted | **Worksheet Progress:** 86%
Potential Grade: 7.30/10.00 (73.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 5/4/2026 3:36:25 PM
Updated: 5/8/2026 11:08:49 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-3/grading/gec23
View Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2026/it202-milestone-3/view/gec23

Instructions

1. Refer to Milestone3 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone3 branch
    1. `git checkout Milestone3` (ensure proper starting branch)
    2. `git pull origin Milestone3` (ensure history is up to date)
5. Fill out the worksheet below
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to GitHub
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone3`
    4. On GitHub merge the pull request from `Milestone3` to `qa`
    5. On GitHub create a pull request from `qa` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the Render.com prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout qa`
11. `git pull origin qa`

Section #1: (3 pts.) Api Data

Progress: 100%

Task #1 (1 pt.) - Concept of Data Association

Progress: 100%

Details:

- What is the concept of your data association to users? (examples: favorites, wish list, purchases, assignment, etc)
- Describe with a few sentences

Your Response:

The data in this application is associated with users through an assignment/ownership concept, where each anime record can be linked to a specific user who created or saved it. This means users can manage their own set of anime entries, such as viewing, editing, or deleting only the records they are responsible for, while admins may have broader control over all records. This approach helps organize data per user, supports personalized interaction, and enforces basic access control so users only modify data relevant to them.

Saved: 5/4/2026 3:36:25 PM

Task #2 (1 pt.) - Data Updates

Progress: 100%

Details:

- When an associated entity is updated (manually or via API), how is the association affected?
  - Does the user see the old version of the data?
  - Does the user see the new version of the data?
  - Does the user need to have data re-associated or remapped?
- Explain why.

Your Response:

When an associated entity is updated, the association to the user is not affected because it is based on a stable identifier (such as the record's id or user_id), which does not change during updates. The user will see the new version of the data, since the update modifies the existing record in place rather than creating a new one. The old version is not shown unless versioning is implemented, which it is not in this case. Because the same record is being updated and its identifier remains the same, the user does not need to re-associate or remap the data—the relationship between the user and the entity stays intact automatically.

Saved: 5/4/2026 3:37:24 PM

Task #3 (1 pt.) - Handling Association

Progress: 100%

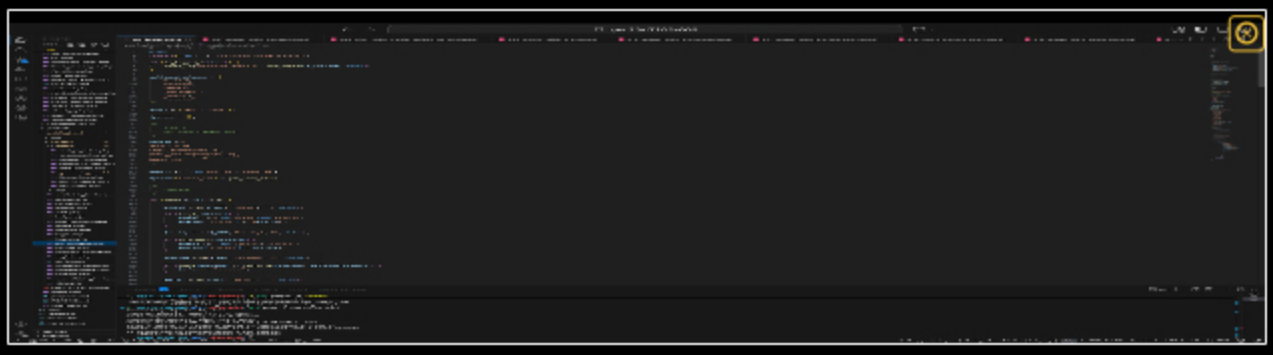
Part 1:

Progress: 100%

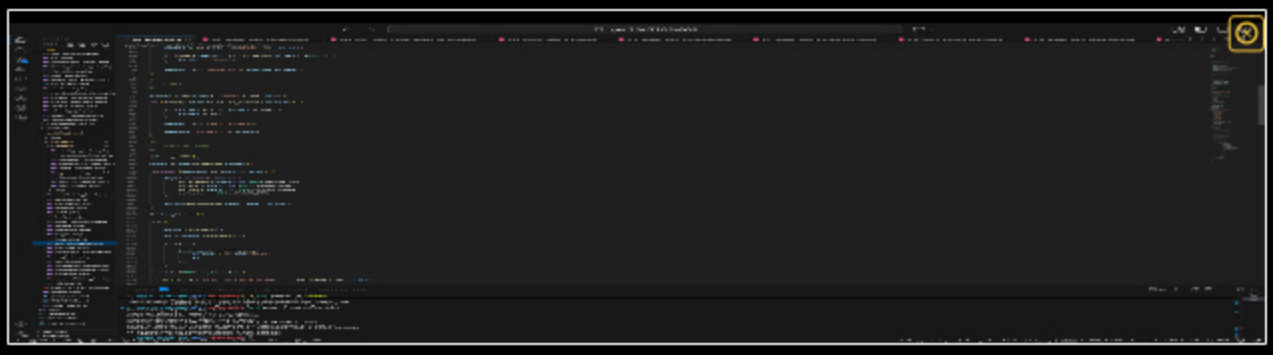
Details:

- Show an example page of where a user can get data associated with them
- Ensure the Render.com qa URL is visible
- Caption if this is a user-facing page or admin page
- Include screenshots of code related to this logic including the database query

this is where the cards would be



code 1



code 2

Saved: 5/8/2026 10:47:16 PM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page that triggers and handles this logic

URL #1
https://gec23-it202-008.onrender.com/project/my_brokers.php

URL
https://gec23-it202-008.onrender.

URL #2
https://github.com/gcruzmedina/gec23-IT202-008

URL
https://github.com/gcruzmedina/

Saved: 5/8/2026 10:47:16 PM

Part 3:

Progress: 100%

Details:

- Describe the process of associating data with the user.
- Can it be toggled, or is it applied once?

Your Response:

The application associates data with users by storing the logged-in user's ID with the generated data in the database. When a user creates an anime card, the system saves the user_id along with the anime information in the IT202_G26_Anime_Cards table. Pages such as "My Cards" use database queries filtered by the current user's ID so users only see their own data. The association is applied whenever new data is created and can later be modified or removed through updates or deletion features.

Saved: 5/8/2026 10:47:16 PM

Section #2: (6 pts.) Associations

Progress: 58%

Task #1 (1.50 pts.) - Logged-in User's Associated Entities

Progress: 100%

Details:

- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a link/button to remove all associations for the particular user
- The page should have a section for stats (number of results and total number possible

based on the query filters)
- The page should have logical options for filtering/sorting
  - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 100%

Details:

- Show a few examples of this page from Render.com qa with various filters applied
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include the code screenshots related to this page

glitches, but the things appear

Saved: 5/8/2026 10:58:11 PM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1
https://github.com/gcruzmedina/gec23-IT20[illegible]

URL
https://github.com/gcruzmedina/(

Saved: 5/8/2026 10:58:11 PM

Part 3:

Progress: 100%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

I implemented the association output by joining the AnimeCards table with the UserAnimeCards relationship table so the page only displays anime cards associated with the currently logged-in user. Each line item is displayed as a Bootstrap card containing a logical summary of the entity, including the title, episodes, score, status, rarity, and power. I added a "View" button that links to the single-view page for the anime card and a "Remove" button that deletes only the relationship between the user and the anime card without deleting the actual entity. I also created a "Remove All Associations" feature that removes all relationships for the logged-in user from the junction table. To satisfy the stats requirement, I displayed both the number of filtered results shown and the total possible associated results. The page includes filtering and sorting options for title, rarity, sort column, and sort order, and I validated these options server-side for security. I implemented a user-controlled limit between 1 and 100 with server-side validation to enforce the allowed range. Finally, when filters return no matching records, the page displays a "No anime cards found" message so the user receives clear feedback.

Saved: 5/8/2026 10:58:11 PM

Task #2 (1.50 pts.) - All Users Association Page

Progress: 33%

Details:

- Each line item should have a logical summary
- Each line item should include the username this entity is associated with
    - Clicking the username should redirect to that user's public profile
- Each line item should include a column that shows the total number of users the entity is associated with
- Each line item should have a link/button to a single view page of the entity
- Each line item should have a link/button to a delete action of the relationship (doesn't delete the entity or user, just the relationship)
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
    - Include a filter for partial username match to show only matching users' associated entities
    - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
    - A filter with no matching records should show "no results available" or equivalent
- When a filter is applied, the page should have a link/button to remove all associations for the matching user(s)

Part 1:

Progress: 0%

Details:

- Show a few examples of this page from Render.com qa with various filters applied
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query

Missing Caption

Saved: 5/8/2026 11:08:38 PM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1
https://github.com/gcruzmedina/gec23-IT202-008

URL
https://github.com/gcruzmedina/

URL #2
https://gec23-it202-008.onrender.com/project/admin/assign_roles.php

URL
https://gec23-it202-008.onrender.

Saved: 5/8/2026 11:08:38 PM

Part 3:

Progress: 0%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

Missing Response

Saved: 5/8/2026 11:08:38 PM

Task #3 (1.50 pts.) - Unassociated Page

Progress: 0%

Details:

- Each line item should have a logical summary
- Each line item should have a link/button to a single view page of the entity
- The page should have a section for stats (number of results and total number possible based on the query filters)
- The page should have logical options for filtering/sorting
  - A limit should be applied between 1 and 100 and controlled by the user (server-side enforces rules)
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 0%

Details:

- Show a few examples of this page from Render.com qa with various filters applied
- Ensure the Render.com qa URL is visible
- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query

Missing Caption

Saved: 4/30/2026 3:25:58 PM

Part 2:

Progress: 0%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1
Missing URL

URL

Saved: 4/30/2026 3:25:58 PM

Part 3:

Progress: 0%

Details:

- Describe how you implemented the association output
- Describe how you solved the various items required for this page (i.e., line item requirements, stats, filter/sort, etc)

Your Response:

Missing Response

Saved: 4/30/2026 3:25:58 PM

Task #4 (1.50 pts.) - Admin Association Page (Like User Roles)

Progress: 100%

Details:

- The page should have a form with two fields
  - Partial match for username
  - Partial match for entity reference (name or something user-friendly)
- Submitting the form should give up to 25 matches of each
  - Likely best to show as two separate columns
- Each entity and user will have a checkbox next to them
- Submitting the checked associations should apply the association if it doesn't exist; otherwise it should remove the association
  - A filter with no matching records should show "no results available" or equivalent

Part 1:

Progress: 100%

Details:

- Show a few examples of this page from Render.com qa with various selections having been submitted
- Ensure the Render.com qa URL is visible

- Ensure each requirement is visible
- Include screenshots of code related to this logic including the database query

admin users

code

Saved: 5/8/2026 11:08:49 PM

Part 2:

Progress: 100%

Details:

- Include the pull request link for this feature
- Include the Render.com prod link to this page

URL #1
https://github.com/gcruzmedina/gec23-IT202-008

URL
https://github.com/gcruzmedina/

URL #2
https://gec23-it202-008.onrender.com/project/admin/assign_roles.php

URL
https://gec23-it202-008.onrender.

Saved: 5/8/2026 11:08:49 PM

Part 3:

Progress: 100%

Details:

- Describe how your code solves the requirements; be clear

Your Response:

I implemented the Admin Association Page by creating a form with partial match filters for usernames and anime titles. The page searches the database and displays up to 25 matching users and anime entities in two separate columns with checkboxes next to each result. When the form is submitted, the code checks if an association already exists in the UserAnimeCards table. If it exists, the association is removed; otherwise, a new association is added. I also added admin permission checks and messages for cases where no matching results are found.

Saved: 5/8/2026 11:08:49 PM

Section #3: (1 pt.) Misc

Progress: 100%

Task #1 (0.33 pts.) - GitHub Details

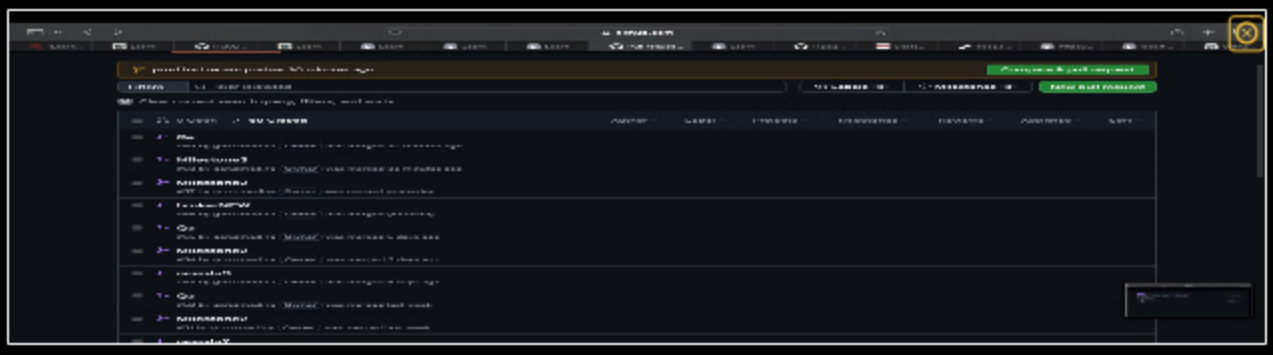
Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history

one



two

Saved: 5/8/2026 10:48:52 PM

Part 2:

Progress: 100%

Details:
Include the link to the pull request for Milestone3 to qa (should end in `/pull/#`)

URL #1
https://github.com/gcruzmedina/gec23-IT202-008

URL
https://github.com/gcruzmedina/

Saved: 5/8/2026 10:48:52 PM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

one

two

Saved: 5/7/2026 9:23:27 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

During this project, I learned how to build a PHP web application using databases, sessions, and role-based access control. I also learned how to debug PHP errors, fix file paths, and connect backend logic with frontend rendering.

Saved: 5/7/2026 9:22:01 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

truly like NOTHING. coding is a whole other ball game idk how yall do it, this was def one of the biggest challenges for me.

Saved: 5/7/2026 9:20:49 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

MAN EVERYTHING. like the logic made sense, and def over time things were making more sense, but overall this was lowkey the hardest one for me. Im glad i got it working.

Saved: 5/7/2026 9:20:18 PM